C02 ASSESSMENT TOOL-1

CSA1305 -TOC

V.Navyasree(192373050)

1.Aim

To design a **Deterministic Finite Automaton (DFA)** over Σ = {0,1} that accepts all binary strings ending with **"01"**, minimize the DFA, and explain its real-world applications.

Theory

In many security systems such as **ATM machines, embedded security devices, login authentication systems, and smart cards**, passwords must satisfy specific patterns.

A DFA is used because it:

- Processes one character at a time.
- Requires only one scan.
- Produces deterministic results.
- Is fast and memory efficient.

Here the password is accepted **only if the last two bits are "01".**

Language

Examples accepted:

- 01
- 101
- 0001
- 1101

Examples rejected:

- 10
- 11
- 00
- 1110

DFA

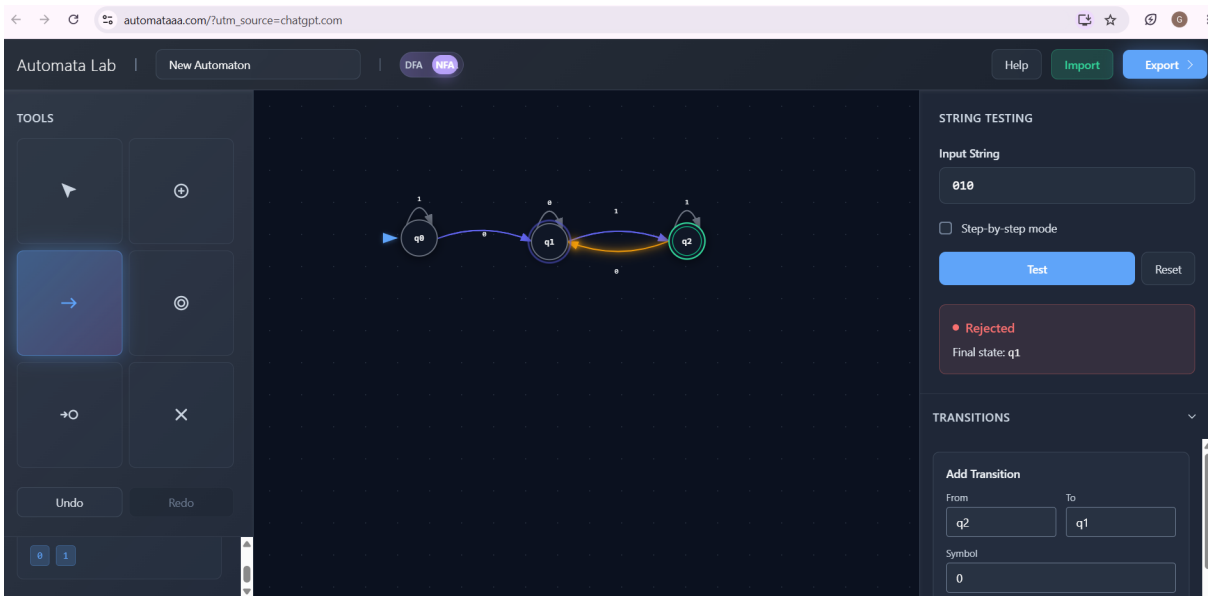
States

- **q0** → Start state
- **q1** → Last symbol is 0
- **q2** → Last two symbols are 01 (Accept)

Transition Table

Current State	Input 0	Input 1
→ q0	q1	q0
q1	q1	q2
*q2	q1	q0

Automata Diagram



Minimization

The DFA already has only **3 states**.

No two states are equivalent.

Therefore,

Minimum DFA = Original DFA

Number of states before minimization = **3**

Number after minimization = **3**

Real-world Applications

- ATM PIN verification
- Password authentication
- Embedded security controllers
- Digital lock systems
- Smart cards
- IoT authentication

Advantages of Minimized DFA

- Less hardware memory
- Faster processing
- Reduced circuit complexity
- Lower power consumption
- High-speed password validation

Conclusion

A minimized DFA validates passwords ending with **"01"** efficiently, making it ideal for authentication systems and embedded hardware.

Regular Expression for "aaaabb"

2.Aim

To design a regular expression for the string **aaaabb** and explain how regular expressions validate user input.

Theory

A **Regular Expression (RE)** is a sequence of symbols used to describe patterns.

Web applications use regular expressions to check whether user input follows the required format.

Examples:

- Email validation
- Mobile number validation
- Password rules
- ZIP codes
- Usernames

Regular Expression

Since only one string is accepted,

aaaabb

or mathematically

Language

Working

Input:

aaaabb

Accepted

Input:

aaabb

Rejected

Input:

aaaabbb

Rejected

Applications in Web Development

Regular expressions validate:

- Email addresses
- Passwords
- Phone numbers
- Usernames
- PIN numbers
- Postal codes
- Credit card formats

Advantages

- Fast validation
- Reduces server load
- Prevents invalid input
- Improves security
- Easy implementation

Role of Regular Languages

Regular languages are recognized using finite automata.

Therefore,

Regular Expression

↓

Finite Automaton

↓

Input Validation

This allows web applications to validate millions of requests efficiently.

Conclusion

Regular expressions provide an efficient method for validating user inputs while improving application performance and security.

3. Convert a*b* into Finite Automaton

Aim

To convert the regular expression

into an equivalent finite automaton and explain their equivalence.

Theory

The expression

means

- Zero or more a's
- Followed by zero or more b's

No **a** can appear after a **b**.

Examples

Accepted

ε

a

aa

bbb

aaabbb

ab

Rejected

aba

ba

baba

abba

DFA

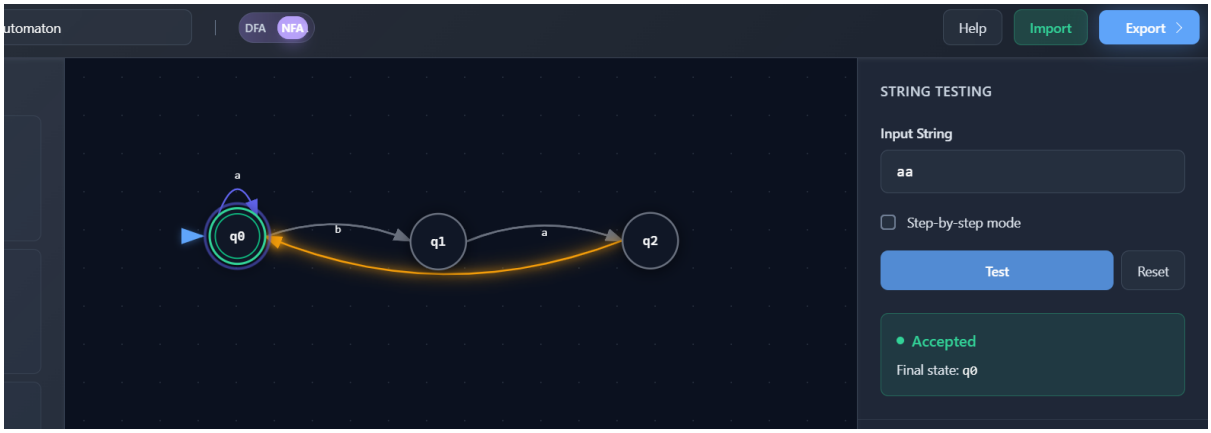
States

- q0 → reading a's
- q1 → reading b's
- q2 → dead state

Transition Table

State	a	b
→*q0	q0	q1
*q1	q2	q1
q2	q2	q2

DFA Diagram



Why RE and FA are Equivalent

Every Regular Expression can be converted into

- NFA
- DFA

Every DFA can be converted back into

- Regular Expression

Hence,

Regular Expression ⇄ Finite Automaton

Both recognize exactly the **Regular Languages**.

Applications

- Compiler lexical analysis
- Intrusion detection
- Log monitoring
- Packet filtering
- Search engines
- Spam filtering

Advantages

- Linear-time processing
- Efficient pattern matching
- Low memory usage
- Easy hardware implementation

Conclusion

Regular expressions and finite automata are mathematically equivalent and are widely used for real-time pattern recognition.

4. Pumping Lemma for L = {$a^n b^n$}

Aim

To prove that

is **not regular** using the Pumping Lemma.

Pumping Lemma

If a language is regular,

then every sufficiently long string

must satisfy

1.
2.
3.

for every

Proof

Choose

where

p = Pumping length

Since

both x and y contain only a's.

Let

where

k>0

Now pump

New string

Now,

Number of a's ≠ Number of b's

Therefore,

This contradicts the Pumping Lemma.

Hence,

is **Not Regular**.

Why Finite Automata Fail

Finite automata have finite memory.

They cannot count equal numbers of

- a's
- b's

Hence they cannot recognize

Implications in Compiler Design

Programming languages contain nested structures such as:

```
if
{
  while
  {
```

}

}

Examples:

- Matching parentheses
- Nested loops
- Function calls
- Balanced braces
- XML tags

Finite automata cannot process these.

Compilers therefore use:

- Pushdown Automata (PDA)
- Context-Free Grammars (CFG)
- Stack-based parsing

Conclusion

The Pumping Lemma proves that is not regular, showing why compilers require more powerful models than finite automata.

5. Closure Properties of Regular Languages

Aim

To explain how closure properties allow multiple pattern-matching rules to be combined while preserving regularity.

Theory

Regular languages remain regular after applying operations called **closure properties**.

This makes them ideal for combining multiple detection rules.

Closure Operations

1. Union (U)

Accepts strings from either language.

Example

Spam OR Malware

2. Concatenation (·)

Combines two patterns sequentially.

Example

Login + OTP

3. Kleene Star (*)

Repeats a pattern zero or more times.

Example

aaaa...

4. Intersection (∩)

Accepts strings common to both languages.

Used in security filters.

5. Complement

Accepts all strings **not** in the language.

Useful for blocking malicious inputs.

6. Difference

Accepts strings in one language but not another.

Used to remove false positives.

Why the Result Remains Regular

Each closure operation can be implemented by constructing another finite automaton.

Therefore,

Regular Language

+

Closure Operation

↓

Another Regular Language

Thus, the resulting language is still recognized efficiently by a DFA or NFA.

Applications

Intrusion Detection

Combine signatures for multiple attack patterns.

Spam Filtering

Detect spam keywords using union and concatenation.

Input Validation

Combine rules for usernames, passwords, and email formats.

Firewalls

Match multiple packet filtering rules.

Lexical Analysis

Recognize identifiers, keywords, operators, and literals together.

Advantages

- Fast execution
- Easy DFA implementation
- Efficient memory usage
- Supports multiple rules
- Scalable for large systems
- Reliable pattern matching

Conclusion

Closure properties enable multiple regular-language patterns to be combined without losing regularity. This guarantees that the combined language can still be recognized efficiently using finite automata, making regular languages highly suitable for applications such as intrusion detection, spam filtering, input validation, and compiler lexical analysis